

Tree / Binary Tree / BST Review Sheet

A tree is where recursion and node pointers merge into one — the linked list and recursion you already practiced come together here. This book walks from manually building a tree all the way to BST deletion. Every function spells out its base case and recursive case, and the questions you asked along the way are collected into a FAQ.

Python 3

Tutor style

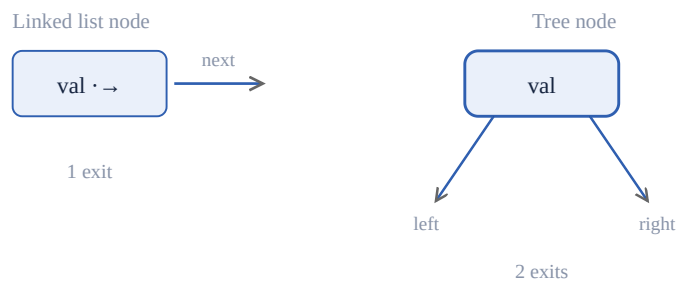
Includes diagrams

Includes FAQ

Includes Java analogies

01 TreeNode & Basic Concepts

A linked list node has only one `next` ; a tree node has two directions, `left` and `right` — that single fork is what grows an entire tree.



One exit vs. two exits — that fork is what turns a line into a tree

TreeNode — nearly identical to Node, just one extra branch

[node](#)

```

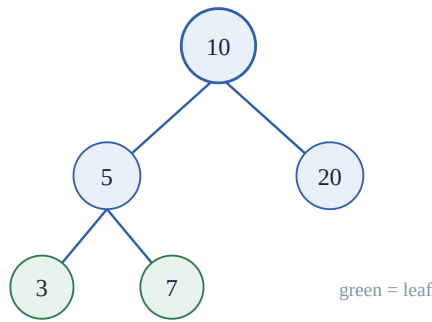
class TreeNode:
    def __init__(self, value):
        self.value = value
        self.left = None    # a linked list has self.next; a tree has one more branch
        self.right = None

```

Vocabulary mapping: a linked list's entry point is called `head` ; a tree's is called `root` . A node with no children is called a **leaf**. Almost every tree function opens with the same line: `if root is None` — an empty tree is recursion's "ground floor," and you plant your feet on it first.

02 Building a Tree by Hand

Learn to wire up a tree by hand first — every function afterward needs something real to run against. This book uses the same sample tree throughout.



The sample tree used throughout this book

Wire it up node by node, by hand

build

```

root = TreeNode(10)
root.left = TreeNode(5)
root.right = TreeNode(20)
root.left.left = TreeNode(3)
root.left.right = TreeNode(7)
# now root.left.left.value == 3
  
```

03 Three DFS Traversals

All three traversals look nearly identical — the only difference is **where** `root.value` sits. Remember: "the position of root = the name of the traversal."

preorder

root → left → right

[10, 5, 3, 7, 20]

inorder

left → root → right

[3, 5, 7, 10, 20]

← sorted for a BST!

postorder

left → right → root

[3, 7, 5, 20, 10]

Same sample tree, three different orders

Three traversals — only the position of `[root.value]` changes

dfs

```

def preorder(root):
    if root is None:                # base case: empty tree returns empty list
        return []
    return [root.value] + preorder(root.left) + preorder(root.right)
    #           ↑ root comes first

def inorder(root):
    if root is None:
        return []
    return inorder(root.left) + [root.value] + inorder(root.right)
    #                               ↑ root comes in the middle

def postorder(root):
    if root is None:
        return []
    return postorder(root.left) + postorder(root.right) + [root.value]
    #                                   ↑ root comes last
  
```

Q I wrote postorder as [3, 5, 7, 20, 10] — what went wrong?

You put root in the **middle** of left and right — that's actually inorder. Look at the subtree `5(3,7)` : postorder must go "left 3 → right 7 → root 5" = `3, 7, 5` . Writing `3, 5, 7` pushes root 5 forward too early. Memory hook: **post = root comes last**, left and right children always print first.

Q Why must `root is None` always `return` instead of directly reading `root.value` ?

An empty node has no `.value` attribute — reading it directly throws `AttributeError: 'NoneType' object has no attribute 'value'`.
Check for empty first, then access is the iron law of every tree recursion.

♦ EXERCISES

1. **Easy** Implement `print_tree(root)` and verify it against all three traversal orders above.

04 `count_nodes / tree_height / search_tree`

Same recursion shape, three different combining rules: `+` for counting, `max` for height, `or` for existence.

count_nodes – count = 1 + left + right

+

```
def count_nodes(root):
    if root is None:           # base case: an empty tree has 0 nodes
        return 0
    return 1 + count_nodes(root.left) + count_nodes(root.right)
    #      ↑ count the current node as 1, plus both subtrees
```

tree_height = 1 + the taller of the two subtrees

max

```
def tree_height(root):
    if root is None:           # empty tree is height 0; a single root is height 1
        return 0
    return 1 + max(tree_height(root.left), tree_height(root.right))
    #      ↑ count the current level as 1, plus the "taller" side
```

Q Why `max` here instead of `+` ?

Height cares about the **longest path** from root to a leaf, not the total number of nodes. If you added the two subtree heights together, you'd get something closer to "total path length across the whole tree," which isn't what height means at all. `count_nodes` wants everything, so it adds; `tree_height` wants the deepest branch, so it takes `max`.

search_tree – return True the moment either side finds it

search

```
def search_tree(root, target):
    if root is None:
        return False
    if root.value == target:
        return True
    return search_tree(root.left, target) or search_tree(root.right, target)
    #      ↑ OR, not AND – either side finding it is enough
```

Q Why `or` in search, not `+` ?

It returns True/False, asking "does it exist anywhere at all." The logic "left has it **or** right has it" is exactly that meaning. A plain binary tree has no ordering rule, so both sides genuinely have to be tried (once you reach BST in § 8, you'll only need to check one side).

♦ EXERCISES

1. **Easy** Following `count_nodes`, write `count_leaves(root)` : count only the leaves (nodes where both left and right are None).
2. **Easy** Write `sum_tree(root)` : the sum of all node values (obviously uses +).
3. **Medium** Write `max_value(root)` : the max value in a plain binary tree (hint: use max to merge left, right, and self).
4. **Medium** Write `count_at_level(root, k)` : return the number of nodes at level k.

05 `BFS Level-Order Traversal`

Everything so far has been recursive (DFS — depth-first search). Level order is the **only one that doesn't use recursion** — it uses a `queue`, sweeping across one level at a time (breadth-first search).

queue over time:

[10]

pop 10

[5,20]

pop 5

[20,3,7]

pop 20

[3,7]

pop 3

[7]

pop 7

[]

Result [10, 5, 20, 3, 7] – every time a node pops, its children get pushed in

level_order – driven entirely by the queue, no recursion

bfs

```
def level_order(root):
    if root is None:
        return []
    result = []
    queue = [root]                # put the root in the queue first
    while len(queue) > 0:
        node = queue.pop(0)       # pop the earliest one in (FIFO)
        result.append(node.value)
        if node.left is not None: # push children onto the tail in order
            queue.append(node.left)
        if node.right is not None:
            queue.append(node.right)
    return result
```

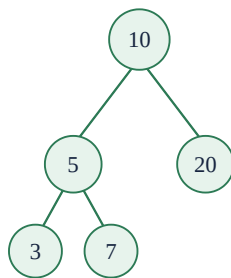
DFS vs BFS in one line: depth-first (DFS, the three traversals) uses **recursion / a stack** – one path all the way to the bottom. Breadth-first (BFS, level order) uses a **queue** – finish one full level before moving to the next. Whenever you see "level by level" or "shortest distance in a tree/graph," think queue.

♦ EXERCISES

- Easy** Implement `level_order` and verify `[10,5,20,3,7]` on the sample tree.
- Medium** Adapt it to group by level, returning `[[10],[5,20],[3,7]]` (hint: record the current queue length at the start of each round).
- Medium** Use BFS to find the tree's maximum width (the most nodes in any single level).

06 is_balanced – Checking Balance

Balanced = **every** node's left/right subtree heights differ by ≤ 1 . Note it's "every node," not just the root.



balanced ✓



unbalanced ✗

```
def is_balanced(root):
    if root is None:
        return True

    left_height = tree_height(root.left)
    right_height = tree_height(root.right)
    if abs(left_height - right_height) > 1:  # check the current node first
        return False

    # key: every subtree must also be balanced
    return is_balanced(root.left) and is_balanced(root.right)
```

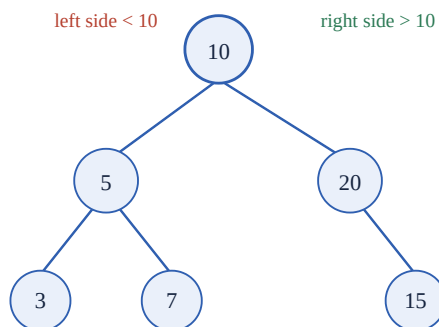
Easy trap: a lot of people only write the `abs(...) > 1` check and return — but that only checks the root. Balance requires **every node** to hit the mark, so you still need `and` to recurse into both left and right's own `is_balanced`. Using `and` here is because we're right back at that truth-table logic — this is an "are they ALL satisfied" kind of check.

♦ EXERCISES

1. **Easy** Run `is_balanced` on both sample trees above — confirm one True, one False.
2. **Medium** Explain why this simple version recomputes height repeatedly (`tree_height` gets called on every node), and think about how to traverse only once.

07 BST Concept

A BST is a binary tree with an ordering rule: **the entire left subtree < root < the entire right subtree**. That rule is what makes lookups fast, $O(\log n)$.



A legal BST: $3 < 5 < 7 < 10 < 15 < 20$ – inorder gives $[3, 5, 7, 10, 15, 20]$, sorted

08 bst_insert / bst_search

Insertion follows the size rule to find where the new node belongs; search follows the same rule and only ever needs to check one side.

bst_insert – recursively find the empty slot and wire it in

insert

```
def bst_insert(root, value):
    if root is None:  # base case: empty slot found, plant the new node
        return TreeNode(value)
    if value < root.value:
        root.left = bst_insert(root.left, value)  # smaller, recurse into left, rewire
    elif value > root.value:
        root.right = bst_insert(root.right, value)  # bigger, recurse into right, rewire
    return root  # hand the (possibly unchanged) root back up
```

Q Why does `root.left` need to be reassigned as `root.left = bst_insert(root.left, value)`, instead of just calling `bst_insert(root.left, value)` ?

When `root.left` is `None`, the recursive call creates a brand-new `TreeNode` and returns it — but that new node only exists inside the function's return value. If you don't **assign it back** to `root.left`, the parent node never actually gets wired to the new node, and the insert silently does nothing.

bst_search — follow the ordering rule, only one side needed

[search](#)

```
def bst_search(root, target):
    if root is None:
        return False
    if target == root.value:
        return True
    if target < root.value:
        return bst_search(root.left, target)    # smaller → only look left
    return bst_search(root.right, target)      # bigger → only look right
```

Compare this with §4's plain-tree `search_tree`: that one needed `or` across both sides because a plain tree has no ordering rule. A BST's rule tells you exactly which side the target must be in, so you only ever recurse one direction — this is precisely why BST search is $O(\log n)$ instead of $O(n)$.

♦ EXERCISES

1. **Easy** Insert `[10,5,20,3,7,15]` one at a time and confirm the resulting tree structure.

09 find_min / find_max

In a BST, the minimum is always found by walking left as far as possible; the maximum by walking right. The iterative version is the more intuitive one.

find_min — all the way left · find_max is its mirror

[min/max](#)

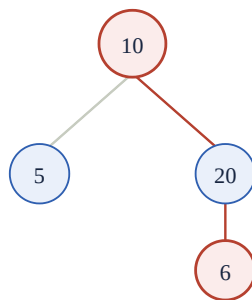
```
def find_min(root):
    if root is None:
        return None
    current = root
    while current.left is not None:    # keep going left as long as there's a left child
        current = current.left
    return current.value

def find_max(root):                    # mirror image: all the way right
    if root is None:
        return None
    current = root
    while current.right is not None:
        current = current.right
    return current.value
```

Why this works: the BST rule guarantees everything on the left is smaller, so "keep going left, never stop" necessarily lands on the smallest value in the whole tree; largest is the mirror case. Both of these functions come back into play in §11 when deleting a node (finding the inorder successor).

10 is_valid_bst — Validation

The trap: **only comparing a node against its parent isn't enough**. Every node must fall inside a "legal range" determined by all of its ancestors, not just its immediate parent.



6 is on 10's right, but < 10 ✗

Locally, $6 < 20$ looks fine – but globally it's illegal: 6 must be > 10

is_valid_bst – carry a (lower, upper) range down to every node

valid

```
def is_valid_bst(root, lower=None, upper=None):
    if root is None:
        return True # base case: an empty tree is valid
    if lower is not None and root.value <= lower:
        return False # broke the lower bound → invalid
    if upper is not None and root.value >= upper:
        return False # broke the upper bound → invalid
    return (
        is_valid_bst(root.left, lower, root.value) and # going left: tighten upper to current value
        is_valid_bst(root.right, root.value, upper) # going right: tighten lower to current value
    )
```

Q I had no idea how to even approach this at first — how should I think about it?

Change the angle: instead of staring at "is parent bigger than child," give **every node a pass** saying `lower < value < upper`. The root's range is unbounded on both sides; going left, everyone must be smaller than their parent, so you tighten `upper` down to the parent's value; going right, you tighten `lower` up to the parent's value. That illegal 6: it's in 10's right subtree, so it inherits `lower=10` — and since $6 \leq 10$, its pass is invalid → False.

11 bst_delete — Deletion

The hardest one in this book. First **find** the node using the size rule, then handle it in **three cases** depending on how many children it has.

bst_delete – find + three cases + rewire

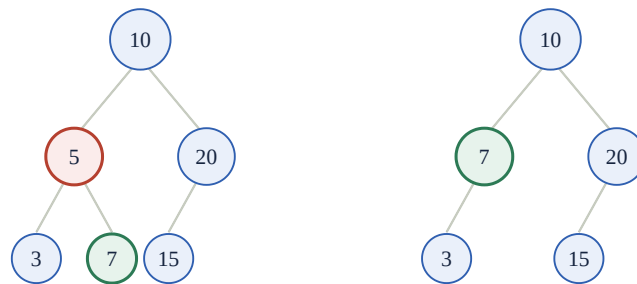
delete

```
def bst_delete(root, value):
    if root is None:
        return None
    if value < root.value:
        root.left = bst_delete(root.left, value) # go delete on the left, wire it back
    elif value > root.value:
        root.right = bst_delete(root.right, value) # go delete on the right, wire it back
    else:
        # found the node to delete
        if root.left is None: # Case 1/2: no left child → right child takes over
            return root.right
        if root.right is None: # Case 2: no right child → left child takes over
            return root.left
        # Case 3: two children → replace with the smallest value in the right subtree (inorder successor)
        successor_value = find_min(root.right)
        root.value = successor_value
        root.right = bst_delete(root.right, successor_value)
    return root
```

CASE	NODE'S CHILDREN	WHAT TO DO
Case 1	Leaf (no children)	Return None, remove it outright
Case 2	Exactly one child	Return that child, let it take the spot
Case 3	Two children	Replace with the right subtree's minimum, then delete that minimum

Q Why must `bst_delete` always `return root` ?

Because deletion can **swap out the root of a subtree**. For example, deleting root 10 when it only has a left child 5 means the new root should be 5. The function must hand the "new root after deletion" back up, so the level above can wire it into the correct place with `root.left = bst_delete(...)`. This is the exact same rewiring logic as insertion and linked-list deletion.



before deleting 5 (two children) after replacing with successor = 7

12 Bug Comparison Table

WRONG	WHY IT'S WRONG	CORRECT
<code>if root.value == None</code>	An empty node has no <code>.value</code> at all — this crashes before the comparison even happens	<code>if root is None</code>
Traversal puts <code>[root.value]</code> in the wrong spot	Root's position in the return expression defines pre/in/post order	Match the position to the traversal's name
<code>tree_height</code> forgets the +1 or the <code>max</code>	Misses counting the current level, or mixes up sum vs. deepest branch	<code>return 1 + max(left, right)</code>
<code>search_tree</code> uses <code>and</code> instead of <code>or</code>	A plain tree has no ordering rule — you need "either side found it"	Use <code>or</code>
<code>root.left = TreeNode(v)</code> in <code>bst_insert</code>	Directly overwrites instead of recursing down to find the right slot	<code>root.left = bst_insert(root.left, v)</code>
<code>is_valid_bst</code> skips lower/upper	Only compares parent-child, misses violations against a farther ancestor	Always thread the range down recursively
<code>bst_insert</code> / <code>bst_delete</code> forget <code>return root</code>	The parent level never gets the rewired subtree back	Every branch must return root (or its replacement)

13 Exercises

Grouped by what you've already practiced, what's new, and what's BST-specific.

♦ ALREADY PRACTICED (REDO WITHOUT LOOKING)

1. `Easy` `count_leaves(root)` / `sum_tree(root)` / `tree_height(root)` / `max_value(root)`.

◆ NEW (PLAIN BINARY TREE)

1. **Medium** `invert_tree(root)` : swap left and right at every node.
2. **Medium** `is_same_tree(a, b)` : check whether two trees are structurally identical with the same values.
3. **Medium** `is_balanced(root)` : implement the single-pass version (avoid recomputing height repeatedly).

◆ BST-SPECIFIC (BUILD WITH [10,5,20,3,7,15] FIRST)

1. **Medium** `is_valid_bst(root)` : verify it correctly rejects the illegal-6 example from § 10.
2. **Medium** `kth_smallest(root, k)` : the k-th smallest value in the BST (hint: inorder traversal gives a sorted list for free).